

RELATÓRIO TÉCNICO

Teste Funcional e Implementação do Programa Identifier

PARTICIONAMENTO EM CLASSES DE EQUIVALÊNCIA

Java 21 | Maven | JUnit 5

Relatório acadêmico - 15 de setembro de 2026

1. Objetivo e especificação

O objetivo deste trabalho é implementar e validar o programa **Identifier** aplicando o critério funcional de particionamento em classes de equivalência, complementado pela análise de valor limite.

Um identificador é considerado válido quando atende simultaneamente às seguintes regras:

1. possui de **1 a 6 caracteres**;
2. começa com uma letra ASCII (**A-Z** ou **a-z**);
3. contém, depois do primeiro caractere, somente letras ou dígitos ASCII (**0-9**).

A entrada nula não pertence ao domínio textual da especificação, mas foi incluída como teste defensivo e deve ser rejeitada.

2. Lista inicial reconstruída

A lista da aula anterior não foi encontrada nos arquivos fornecidos. Para viabilizar a comparação solicitada, foi reconstruída a lista intuitiva abaixo, sem aplicação formal de uma técnica de teste.

Caso	Entrada	Esperado	Motivação intuitiva
CT-A1	abc	Válido	Exemplo comum
CT-A2	1abc	Inválido	Começa com dígito
CT-A3	ab#1	Inválido	Possui caractere especial
CT-A4	abcdefg	Inválido	Excede o tamanho máximo
CT-A5	String vazia	Inválido	Não possui caracteres

A lista cobre regras relevantes, mas não evidencia sistematicamente todas as classes nem os dois lados dos limites de comprimento.

3. Passo 1 - Classes de equivalência

Cada classe reúne entradas tratadas de maneira equivalente pelo programa. Uma entrada representativa de cada classe permite reduzir redundância sem perder cobertura das regras.

ID	Condição	Classe	Domínio	Representante
CE-C1	Comprimento	Válida	1 a 6 caracteres	abc123
CE-C2	Comprimento	Inválida	0 caracteres	String vazia
CE-C3	Comprimento	Inválida	7 ou mais	abcdefg
CE-I1	Inicial	Válida	Letra ASCII	Ab12
CE-I2	Inicial	Inválida	Dígito ASCII	1abc
CE-I3	Inicial	Inválida	Símbolo, espaço ou outro	_abc
CE-F1	Demais	Válida	Letras e dígitos ASCII	a1B2
CE-F2	Demais	Inválida	Símbolo, espaço ou outro	ab#1

As classes CE-I2 e CE-I3 são separadas porque representam naturezas diferentes de caractere inicial inválido. CE-F2 cobre qualquer caractere que não seja letra ou dígito.

4. Passo 2 - Casos derivados

Caso	Entrada	Classes exercitadas	Esperado
CT-E1	a	CE-C1, CE-I1	Válido
CT-E2	abc123	CE-C1, CE-I1, CE-F1	Válido
CT-E3	String vazia	CE-C2	Inválido
CT-E4	abcdefg	CE-C3	Inválido
CT-E5	Ab12	CE-I1, CE-F1	Válido
CT-E6	1abc	CE-I2	Inválido
CT-E7	_abc	CE-I3	Inválido
CT-E8	a1B2	CE-F1	Válido
CT-E9	ab#1	CE-F2	Inválido
CT-E10	null	Robustez	Inválido

5. Análise de valor limite

A análise verifica exatamente as fronteiras do intervalo permitido e os valores imediatamente externos, onde erros de comparação são mais prováveis.

Tamanho	Entrada	Posição	Esperado
0	String vazia	Abaixo do mínimo	Inválido
1	a	Mínimo válido	Válido
6	abc123	Máximo válido	Válido
7	abcdefg	Acima do máximo	Inválido

6. Automação com JUnit 5

Os testes automatizados deixam explícita a estrutura formal solicitada: **Setup** cria o validador ou captura a saída padrão; **Invocation** chama o método de validação ou a aplicação; e **Assessment** compara o resultado obtido com o esperado usando `assertEquals`.

Foram executados 14 casos sobre o validador, incluindo classes, valores-limite e entrada nula, além de 3 casos sobre a interface de linha de comando.

Comando de execução

```
.\mvnw.cmd test
```

7. Resultado

17 TESTES APROVADOS - BUILD SUCCESS

Métrica	Resultado
Testes executados	17
Falhas	0
Erros	0
Ignorados	0
Data da execução	15/09/2026

8. Conclusão

Todos os representantes selecionados produziram os resultados esperados. As classes de equivalência tornaram a seleção rastreável em relação à especificação, enquanto os tamanhos 0, 1, 6 e 7 verificaram precisamente as fronteiras. A aplicação final atende às regras definidas e sua execução automatizada foi concluída sem falhas.

Fonte: especificação da atividade fornecida na pasta **docs** do projeto.